

Quantum Computing: A Brief Introduction and Its Applications in Spatial Computing

Amr Magdy^{1,2}

¹Department of Computer Science
and Engineering

²Center for Geospatial Sciences
University of California, Riverside
Riverside, CA, USA
amr@cs.ucr.edu

Moustafa Youssef

Department of Computer Science and
Engineering

The American University in Cairo
Cairo, Egypt
moustafa-youssef@aucegypt.edu

Ibrahim Sabek^{3,4}

³Department of Computer Science
⁴Spatial Sciences Institute

University of Southern California
Los Angeles, CA, USA
sabek@usc.edu

Abstract

Quantum computing has shown significant potential for addressing complex optimization and computationally intensive problems across a range of domains. Its adoption in spatial data applications presents substantial opportunities for advancing spatial computing. This tutorial provides an in-depth introduction to quantum computing, covering both the quantum circuit model (gate model) and quantum annealing, and discusses their applications in spatial computing. The tutorial is organized into three parts. The first part introduces fundamental concepts in quantum computing. The second part presents quantum gates, highlighting examples of basic gates and their use in circuits. The third part introduces quantum annealing, including Quadratic Unconstrained Binary Optimization (QUBO) modeling and hybrid solvers that combine the power of Quantum Processing Units (QPUs) with classical CPUs. The tutorial also points readers to key resources at the intersection of quantum computing and spatial computing.

CCS Concepts

• **Hardware** → **Quantum computation**; • **Computer systems organization** → **Quantum computing**; • **Mathematics of computing** → **Combinatorial optimization**; • **Computing methodologies** → *Modeling methodologies*.

Keywords

Spatial Computing, Quantum Computing, Quantum Circuits, Quantum Gates, Quantum Interference, Quantum Algorithms, Superposition, QUBO, Quantum Annealing, Combinatorial Optimization

ACM Reference Format:

Amr Magdy^{1,2}, Moustafa Youssef, and Ibrahim Sabek^{3,4}. 2025. Quantum Computing: A Brief Introduction and Its Applications in Spatial Computing. In *The 1st ACM SIGSPATIAL International Workshop on Quantum Computing for Spatial Data Systems and Applications (Q-Spatial '25)*, November 3–6, 2025, Minneapolis, MN, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3764923.3777406>



This work is licensed under a Creative Commons Attribution 4.0 International License. *Q-Spatial '25*, Minneapolis, MN, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2186-1/2025/11

<https://doi.org/10.1145/3764923.3777406>

1 Introduction

Quantum computing is emerging as a new computational paradigm with the potential to tackle problems that are difficult to solve efficiently using classical hardware, particularly those involving large combinatorial search spaces and computationally intensive optimization. This promise has already motivated substantial progress in quantum algorithms, software stacks, and early-stage quantum hardware, and it is increasingly relevant to *spatial computing*; the broad umbrella that includes spatial data management, geospatial analytics, geographic information systems (GIS), remote sensing, and location-based optimization. Many core spatial computing workloads reduce to discrete optimization problems (often NP-hard) with objective functions and constraints driven by distance, proximity, contiguity, coverage, and network structure. These characteristics make spatial applications natural candidates for quantum computing approaches.

This tutorial serves as a self-contained introduction to quantum computing for the spatial computing community. To maximize accessibility and reusability, we provide recordings of the tutorial sessions through the Q-Spatial Workshop online outlets [34, 48, 49]. The tutorial content is organized into three parts. The first part covers quantum computing fundamentals [46], including qubits, superposition, and measurements, and a brief intuition behind quantum speedups. The second part introduces the quantum gate model [47], highlighting representative single- and multi-qubit gates, circuit building blocks, and illustrative circuits that appear in quantum algorithm design. It also highlights the usage of quantum computing in location tracking and localization applications. The third part focuses on quantum annealing [35], introducing Quadratic Unconstrained Binary Optimization (QUBO) modeling and practical hybrid workflows that combine Quantum Processing Units (QPUs) with classical CPUs. A dedicated hands-on session demonstrates quantum programming in practice using the D-Wave quantum annealer [2, 3].

Beyond presenting the computing models, a central goal of this tutorial is to connect quantum methods to concrete spatial computing tasks. A wide range of spatial analysis and geospatial decision-making problems can be expressed as optimization formulations that align naturally with quantum-inspired and quantum-native solvers [4, 5, 19, 28, 29, 36–44, 50, 51]. Representative examples include: facility location problems (e.g., p -median, p -center and covering variants problems); assignment problems (symmetric/asymmetric/side-constrained matching of

demand points to facilities under travel costs); quadratic assignment problems (placing facilities when costs depend on pairwise distances or interactions); graph coloring problems (classic map coloring and zone labeling); set partitioning and set packing problems (e.g., districting and vehicle routing (VRP) formulations, as well as coverage models); maximum cut problems (e.g., image/terrain segmentation and network partitioning); maximum independent set and maximum clique problems (e.g., facility placement and interference/coverage in geometric graphs); discrete tomography problems (remote sensing and image reconstruction); maximum diversity problems (selecting spatially dispersed sites or sensors); clique partitioning problems (clustering when edges arise from spatial proximity); constraint satisfaction problems (CSPs) arising in map labeling and spatial contiguity/adjacency constraints; and task allocation problems where agents and tasks are geolocated. These exemplars highlight the breadth of spatial computing workloads that can benefit from quantum modeling, either by accelerating search, improving solution quality under tight time budgets, or enabling new hybrid solution strategies.

In the remainder of this paper, we introduce the essential background (Section 2) needed to understand quantum computation for the quantum circuit model (Section 3) and quantum annealing (Section 4), provide practical guidance for mapping spatial problems to quantum-ready formulations (especially QUBO), and point readers to curated resources and recordings that provide further detailed explanations, learning material, and use cases that combine quantum computing and spatial computing.

2 Fundamental Concepts

This section introduces basic concepts in quantum computing.

Qubit. A *quantum bit* (qubit) is the basic unit of quantum information (or digital quantum information) and is analogous to the classical bit. The classical bit, i.e., binary digit, can be in only one of two states at any time, taking the value 0 or 1, but not both. Contrary to classical bits, a qubit can exist in a *superposition* of the two states (zero and one states simultaneously). This superposition allows quantum computations to work on both states at the same time, a phenomenon often referred to as *quantum parallelism*. Qubits can have various physical implementations, e.g., the polarization of photons or the spin of electrons.

Quantum superposition is the odd-but-powerful natural quantum rule that allows a single particle, e.g., an electron or a photon, to exist in two possible states at the same time, like a spinning metal coin that fluctuates in states (for both heads and tails) until you look at it, then it settles on one specific state. Quantum superposition is a general phenomenon in natural quantum systems and is not specific to computing systems. In quantum computing, the quantum states are binary, either 0 or 1, to allow quantum computers to implement binary qubits. The power of superposition allows a small collection of qubits to explore many calculation paths in parallel and achieve promising speed-ups on certain problems once the result is finally “*observed*” and collapses to a single answer.

Mathematically, the superposition is modeled as a *wave function* of possibilities that represent the probabilities of the qubit to exist in each distinct state [32]. The wave function is governed by Schrodinger equation, the quantum counterpart of Newton’s

second law of motion in classical physics. Typically, a qubit in a superposition state $|\psi\rangle$ is represented in a two-dimensional Hilbert space $\mathcal{H}$ by the Dirac notation (or bra-ket notation) as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

Where α and β are called the *probability amplitudes* of the classical states $|0\rangle$ and $|1\rangle$, respectively, α and β are complex numbers in the general case. The probability amplitudes represent probabilities of $|\alpha|^2$ and $|\beta|^2$ for a qubit to exist in states $|0\rangle$ and $|1\rangle$, respectively, where $|\alpha|^2 + |\beta|^2 = 1$.

Quantum measurement. When an “*observer*” device measures (or looks at) a qubit, this act of measurement counts as a physical interaction that forces the qubit’s spread-out wave of possibilities to snap into one specific outcome, a sudden change known as wave-function collapse, and mathematically represented by a Dirac delta function. Until that instant the particle lives in a superposition, meaning it simultaneously embodies all the states, e.g., places or spins, it could occupy. However, the act of checking (i.e., measurement) wipes out that blur and leaves the particle with a single, definite result every time, with a probability corresponding to the destructed wave function. The design of a quantum algorithm is responsible for ensuring that the correct answer is obtained with high probability. Further details about guiding the correct answer and suppressing incorrect answers are discussed in Section 3.

Quantum speedups come from the fact that quantum information behaves like a *wave* rather than a single deterministic value. A quantum computer can place a collection of candidate solutions into a superposition, apply operations that make their probability amplitudes evolve coherently, and then recombine those amplitudes so that desirable outcomes are *amplified* while undesirable ones are *suppressed*. This is not “trying all answers at once” in the classical sense, as quantum measurement reveals only one outcome, but it is the ability to shape how probability flows through the space of possibilities using interference (see quantum interference in Section 3). Two additional ingredients often enable advantage: *entanglement*, which correlates many variables so the state cannot be represented as independent parts, and *problem structure*, which quantum algorithms exploit to steer amplitude toward solutions using fewer steps than the best-known classical approaches.

Our tutorial, presented in [46], provides further explanation of the fundamental concepts of quantum computing.

3 Quantum Circuit Model

Quantum computing has two major models in practice: (1) the quantum circuit model, or quantum gate model, and, (2) quantum annealing. The quantum annealing is introduced in Section 4. In the quantum gate model, operations on qubits are represented by *gates*, similar to gates in a classical logic circuit. An example of a common quantum gate is the NOT gate (also called the Pauli-X gate or X gate) that is analogous to the classical NOT gate. Specifically, when we apply the NOT gate to the state $|\psi_0\rangle = \alpha|0\rangle + \beta|1\rangle$, we obtain a flipped state $|\psi_1\rangle = \beta|0\rangle + \alpha|1\rangle$. So, if the qubit is in a classical state zero, i.e., $\alpha = 1$, $\beta = 0$, and $|\psi_0\rangle = |0\rangle$, applying the NOT gate $X|\psi_0\rangle$ gives $|\psi_1\rangle = X|\psi_0\rangle = X|0\rangle = |1\rangle$, and similarly for $|1\rangle$.

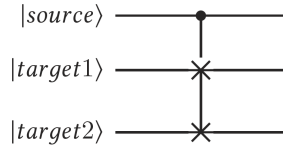


Figure 1: Example of controlled gates: the two target qubits are swapped iff the control qubit is $|1\rangle$.

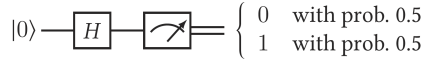


Figure 2: A simple quantum circuit. Single lines carry quantum information while double lines carry classical information [38]

Gates are usually represented by unitary matrices while states are represented by column vectors.¹ The matrix for the NOT gate is $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ and the operation above can be written as $|\psi_1\rangle = X|\psi_0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix} = \beta|0\rangle + \alpha|1\rangle$.

Another important gate is the Walsh–Hadamard gate (H gate), which maps $|0\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, i.e., a superposition state with equal probability for $|0\rangle$ and $|1\rangle$, and maps $|1\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

The matrix for the Hadamard gate is $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ and the Hadamard operation can be written as $|\psi_1\rangle = H|\psi_0\rangle$.

Both X and H gates are single-qubit gates, i.e., take only one qubit as input. Other quantum gates take multiple qubits as input and are called multi-qubit gates, such as the CNOT and CSWAP gates. Figure 1 illustrates a common multi-qubit gate, the *controlled* gate. In such a gate, the operation (e.g. SWAP) is performed on the target wire(s) *iff* the control qubit is in state $|1\rangle$. Controlled operations are often used to *entangle* qubits, meaning measurements on one qubit reveal information about the other, even when spatially separated. Our tutorial presented in [47] provides further details on various gate operations and example famous quantum circuits, including the quantum entanglement circuit.

Example quantum circuit. Figure 2 shows a simple quantum circuit. Single lines carry quantum information while double lines carry classical information (typically after measurement). The circuit applies an H gate to state $|0\rangle$, producing the state $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Upon measurement, the result is either 0 or 1 with equal probability, and the state collapses to the observed classical bit value.

Quantum interference. The probabilistic nature of quantum circuits is a key difference compared to classical logic circuits. This probabilistic nature requires new mechanisms to control the outcomes. At the core of quantum computing, quantum interference mechanisms are used to cleverly bias the amplitudes of the qubits toward the desired state, thereby achieving a specific computational result. Quantum interference leverages the wave-like properties

of qubits to amplify correct answers and cancel incorrect ones, acting like noise-canceling headphones for computation. Such manipulation enables quantum algorithms to find correct solutions by steering the probability distribution toward desired outcomes through constructive (amplifying) and destructive (canceling) interactions of waves. This manipulation of probabilities is crucial for quantum speedup, enabling complex calculations by increasing the likelihood of measuring desired states. However, it requires extreme control to avoid unwanted external interference (decoherence) that destroys quantum states.

Quantum interference is implemented through quantum gates. Three relevant concepts for understanding quantum interference at a high level are *Constructive Interference*, *Destructive Interference*, and *Decoherence Time* (or *Dephasing Time*) [32], briefly outlined below.

Constructive interference. Figure 3 shows an example of constructive wave interference. When waves of two or more qubits overlap in phase, i.e., their peaks align with each other and so their valleys, they combine and create a new larger wave with an amplitude equal to the sum of the individual waves' amplitudes. From a quantum computing perspective, this increases the probability amplitudes of the new state associated with the desired outcome, thereby increasing the likelihood of measuring that outcome.

Destructive interference. Figure 3 shows examples for wave destructive inference. On the contrary to constructive interference, the destructive interference of waves associated with two or more qubits happens when they overlap out of phase, i.e., peaks of some waves align with valleys of other waves and vice versa. In such cases, waves destructively interfere, and the resulting wave has a much smaller amplitude than the individual waves. From a quantum computing perspective, this decreases the probability amplitudes of the new state associated with the wrong outcome, thereby decreasing the likelihood of measuring that outcome. As shown in Figure 3, the destructive interference is either partially or fully destructive. In partially destructive interference, the phase shift is limited, and so the resultant wave has a smaller amplitude but it still larger than zero, i.e., the two waves do not cancel each other. In fully destructive interference, the phase shift is so significant that the two waves completely cancel each other and the resultant wave has zero amplitude. In quantum computing, the zero amplitude corresponds to zero probability to produce such state, i.e., completely eliminate an undesirable output.

Quantum inference can be further illustrated through several layperson analogies to better understand how a quantum algorithm or circuit design uses constructive and destructive interference to guide the correct output and eliminate the incorrect outputs. One possible analogy for a quantum circuit in a superposition is a sports stadium filled with thousands of people shouting many different phrases simultaneously, each phrase is an analogy for a possible answer. An orchestrator (a quantum algorithm) tries to pick *one correct answer* (phrase) from a huge crowd of possibilities, but cannot directly "hear" (observe or measure) without ruining the process. So, the orchestrator uses a clever voting-by-overlap trick that amplifies the right chant, cancels the wrong ones. He can do two things repeatedly: (1) Give the correct chant a tiny timing advantage, so everyone chanting the correct phrase is told to clap slightly off-beat

¹The ket notation $|*\rangle$ denotes column vectors, while the bra notation $\langle*|$ denotes row vectors.

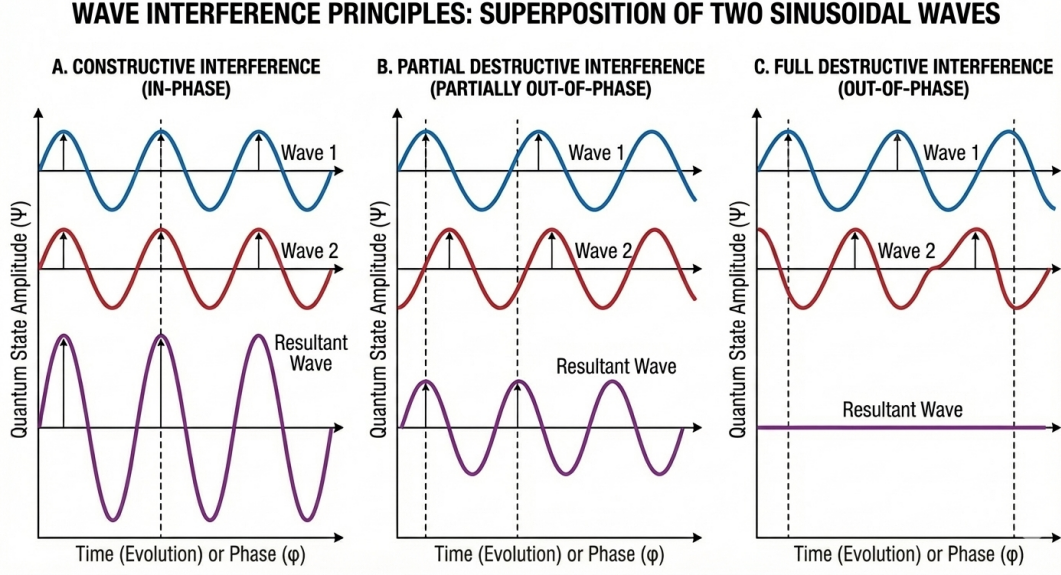


Figure 3: Constructive vs. Destructive Quantum Wave Interference.

compared to everyone else. This is like giving the correct quantum state a different phase. (2) Run a mixing clap that makes chants interfere. So, the orchestrator leads a sequence of actions where claps and voices overlap in the stadium. When two groups' sound waves line up, they get *louder* (constructive interference). When they line up opposite, they *cancel out* (destructive interference). After a few rounds, the people chanting the correct phrase become increasingly in sync with each other, so their sound keeps getting amplified. On the contrary, the people chanting wrong phrases end up out of sync in lots of different ways, so their sounds cancel one another more and more. When an observer "hear the stadium" (measure) after orchestration, he is overwhelmingly likely to hear the one loud dominant chant, which is the correct answer. Similar mechanisms are used for audio noise cancellation techniques.

Decoherence time, also known as *dephasing time*. While quantum interference is desired, environmental noise, e.g., heat, radiation, or electromagnetic fields, acts as unwanted interference, causing qubits to lose their quantum state (decoherence). The dephasing time is the time for how long a qubit maintains its phase coherence, meaning its ability to exist in superposition states keeping the same values of probability amplitudes α and β before these phases get lost due to environmental noise. The dephasing time is a key measure of qubit quality and quantum circuit ability to do complex computations. The shorter dephasing time means faster loss of quantum information, limiting complex computations. All computations within the quantum circuit must finish before the circuit dephasing time.

Quantum register. The notion of a qubit can be extended to higher dimensions via a *quantum register*. A register $|\psi\rangle$ consisting of n qubits lives in a 2^n -dimensional complex Hilbert space $\mathcal{H}$. For example, $|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle$ with $\sum_i |\alpha_i|^2 = 1$. Basis state $|i\rangle$ denotes the n -bit binary encoding of the integer i .

We use the tensor product $\otimes$ to compose quantum systems. For instance, combining $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and $|\phi\rangle = \gamma|0\rangle + \delta|1\rangle$ yields $|\omega\rangle = |\psi\rangle \otimes |\phi\rangle = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$.

Quantum algorithm. A convenient way to describe a quantum algorithm is via a *quantum circuit*, which in the general case is a collection of gates applied to quantum registers. The circuit manipulates the quantum system state using superposition, entanglement, and interference to reach a desired output state that encodes the algorithm's answer. The final step is to measure the output state(s), revealing the required classical information.

Our tutorial presented in [47] provides examples of well-known quantum algorithms and circuits, including quantum random generation, quantum entanglement, quantum teleportation, quantum location tracking, quantum cosine similarity, and Bell circuits.

Finally, the *no-cloning theorem* states that quantum bits cannot be copied. Consequently, quantum information cannot be duplicated as in classical systems, which has important implications for cryptography (preventing perfect eavesdropping) while making classical error-correction techniques unsuitable for quantum states.

4 Quantum Annealing

4.1 Preliminaries

Overview. The quantum circuit model introduced in Section 3 is a universal computing model that, in theory, can solve any computable problem. The other major model of quantum computing is quantum annealing (QA) that is based on adiabatic quantum computation (AQC) [21]. It has been shown that the quantum circuit model and adiabatic quantum computation are polynomially equivalent in theory [1, 31]. QA has been proposed as a novel heuristic for combinatorial optimization problems [23], and it is similar to its classical counterpart, Simulated Annealing (SA) [26, 27]. In SA, assuming the objective function is minimized, the configuration

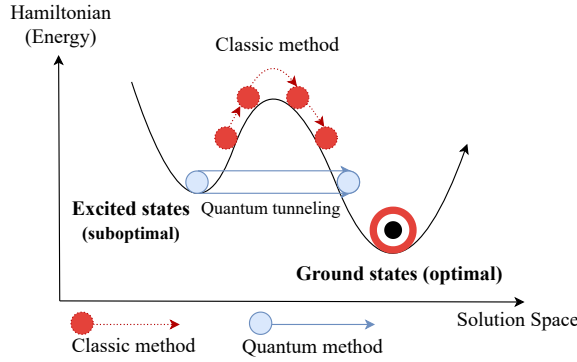


Figure 4: Quantum Annealing Process vs. Classic Method [5]

space is explored by randomly proposing changes to the state: if the new objective function value is lower, the change is accepted; if the objective function increases, the change is accepted with some probability. In QA, these changes are applied to a quantum superposition of configurations, as briefly explained in [33], enabling massively parallel search of the configuration space. This allows phenomena such as *quantum tunneling* to facilitate exploration, potentially avoiding becoming trapped in local minima.

Unlike classical simulated annealing, which is based on thermal fluctuations, QA uses quantum tunneling to escape local optima and potentially converge to the global optimum. This process involves initializing a quantum system in a superposition that encodes a quadratic objective function over binary variables via interactions between two-level quantum mechanical systems, i.e., qubits, so that the solution to the problem of interest is the configuration that minimizes the energy of the interacting system. The annealing process begins with a superposition of all possible configurations and then gradually changes the interaction strengths between the qubits. This annealing process (if performed slowly enough) transforms the initial superposition into a state that minimizes the system's energy, thereby encoding the solution to the combinatorial optimization problem [28, 29, 36].

Figure 4 shows an example for a solution space for an arbitrary NP-hard optimization problem represented by an energy curve, called the Hamiltonian. Every possible solution of the NP-hard problem is a point on the x-axis of that curve, and its solution quality corresponds to a point on the y-axis representing the overall energy of the underlying quantum system. The Hamiltonian [6, 30, 33] comprises multiple local maxima (hills) and minima (valleys), each corresponding to a different candidate solution for that NP-hard problem. The optimal solution corresponds to the deepest valley, depicted as nested circles in Figure 4 representing the ground state. In that solution space, a transverse field functions as a subtle fluctuating perturbation that allows the system to quantum tunnel like a marble through energy barriers rather than surmounting them via classical thermal activation [26]. In Figure 4, reaching from a suboptimal solution of the NP-hard problem (an excited state represented by the first valley) to the optimal solution (a ground state represented by the second valley), a classical probabilistic method

must surmount the hill and sample many states in between. In contrast, the quantum annealer can approach the optimal solution while exploring a significantly smaller number of states through quantum tunneling. By carefully controlling this push, following an adiabatic trajectory, the marble gradually finds its way to the deepest valley, which corresponds to the ground state of the Hamiltonian, the optimal solution [7]. Quantum annealing can be applied to a broad range of NP-hard problems that traditionally depend on classical heuristic methods. Notably, several companies have developed hardware that implements a quantum annealer aimed at addressing a variety of optimization challenges [20].

Our tutorial, presented in [35] with hands-on examples in [2, 3], is based on the D-Wave quantum annealer. The D-Wave quantum annealer [22, 24, 25, 27] implements the quantum annealing Ising model [14] using superconducting technology, where small superconducting loops represent individual qubits: current circulating around these loops in both directions encode the 0 and 1 states of the qubit. For two qubits that are close to each other, a programmable interaction can be implemented between them, as well as local biases on the individual qubits. With these elements, an interacting system can be constructed whose energy takes the form

$$E(\vec{s}) = \sum_{ij} J_{ij} s_i s_j + \sum_i h_i s_i \quad (2)$$

where $\vec{s} = (s_1, \dots, s_n)$ is a binary vector representing the system configuration, and (h_i, J_{ij}) are parameters representing the local biases on each qubit, and the interaction strengths between them. From a high-level application perspective, one can see each variable s_i as a binary decision variable, a binary choice for a certain application-level option, $s_i \in \{-1, +1\}$, and the Hamiltonian $E(\vec{s})$ uses h_i and J_{ij} as a numbers set to reward or penalize certain choices and pairwise combinations. Minimizing $E(\vec{s})$ (getting the lowest score) gives the preferred configuration that represents the optimal solution of this Hamiltonian.

QUBO. Unlike the quantum circuit model, the quantum annealer does not use quantum gates to design and execute quantum algorithms. Instead, it employs Quadratic Unconstrained Binary Optimization (QUBO) formulations to model optimization problems mathematically. Then, QUBO formulations are coded using programming tools, e.g., the Ocean SDK [9, 13], and mapped directly to the quantum annealer's Quantum Processing Unit (QPU). One can view QUBO formulations as the lowest-level representation executable by a QPU, analogous to binary machine code on classical CPUs, but they are programmed in high-level languages such as Python [13]. Section 4.2 gives an overview about QUBO formulations that are further detailed in our tutorial [35] and hands-on exercises [2, 3].

Hybrid solvers. QUBO formulations run natively on quantum processing units (QPUs). However, current QPUs still have limited capacity with a maximum of 5000 qubits and 20-way connected qubits architecture [10, 15]. This triggered a need to develop hybrid solutions that combine the usage of both QPUs with classical CPUs. Our tutorial is based on D-Wave hybrid solvers. D-Wave Quantum Inc. currently provides four hybrid solvers, namely, BQM [16], DQM [12], CQM [17], and NL solver [18]. Section 4.3 provides an overview about different hybrid solvers that are further detailed in our tutorial [35].

Limitations of quantum annealing. One of the main limitations of quantum annealers is the restriction imposed on the available interactions due to the geometrical constraints imposed by the actual physical layout of the qubits. Ideally, we would like all qubits to be able to interact with every other qubit. However, to make two qubits interact, they must be close together, and when we lay out the physical superconducting loops on a two-dimensional surface, we see that this requirement is impossible to fulfill. Hence, each qubit can only interact with a fixed number of other qubits, resulting in a sparse connectivity graph. For the D-Wave Advantage system [45] that we have access to (that uses the Pegasus architecture [15]), the maximum degree of this graph is 15. The most advanced D-Wave system is Advantage2 system (that uses the Zephyr architecture [10]), the maximum degree is boosted to 20. Even though there are techniques to get around this issue by mapping a single variable to a chain of physical qubits that has a larger effective connectivity degree, we pay a price in the size of problems that can be implemented (since the total number of qubits in the processor is fixed). Encoding other problem features like constraints further reduces the number of qubits available to encode variables. However, as we shall see next, there are other ways of exploiting the quantum mechanical features of the processor by combining them with classical approaches.

4.2 Quadratic Unconstrained Binary Optimization (QUBO)

Quantum processing units (QPUs) of quantum annealers execute Quadratic Unconstrained Binary Optimization (QUBO) formulations natively [8]. QUBO is a type of mathematical optimization methods that models complex real-world optimization problems, like logistics, localization, or finance, by minimizing a *quadratic* objective function of *binary* (0/1) decision variables. Thus, every QUBO formulation must define two components: (1) a set of binary variables, and (2) an objective function that has all its terms quadratic, i.e., a multiplication of exactly two decision variables, allowing one free term, i.e., constant. The QUBO binary decision variables are suitable for binary computing systems, e.g., qubits. QUBO formulations are, by default, unconstrained; therefore, any constraints on the decision variables are enforced via additional penalty terms added to the quadratic objective function. The more additional penalty terms added, the more required interactions between qubits at the hardware level, which impact the solvable problem size as discussed in the limitations in Section 4.1. The main advantage of QUBO formulations is that they transform any problem into a universal format, which reduces diverse optimization challenges to the task of finding the lowest "energy state" of a system, thereby bridging classical computation and quantum hardware.

A QUBO formulation consists of: (1) a set of binary decision variables, and (2) a quadratic objective function (to minimize). As a simple illustration of how QUBO formulations are constructed, we consider a trivial optimization problem: buying ice cream scoops at minimal cost. Assume a client has two ice cream flavors, one vanilla scoop for \$2 and one chocolate scoop for \$5, and the task is to buy *at most one scoop* at the minimum cost. The trivial optimal solution is buying none at \$0. To represent this simple optimization problem

in a QUBO formulation, we use two decision binary variables: x_1 for buying a vanilla scoop and x_2 for buying a chocolate scoop. Our objective function f_1 consists of two terms reflecting the cost of each scoop as follows:

$$f_1 = 2x_1 + 5x_2 \quad (3)$$

f_1 represents the dollar amount of buying the ice cream scoops. However, both terms in f_1 are linear, not quadratic, which, at first glance, contradicts the definition of the QUBO formulation. However, closely looking into the properties of a binary variable, for any binary variable x , the value of x equals the value of x^2 , by enumeration of values, i.e., at $x = 0, x^2 = 0$ and at $x = 1, x^2 = 1$. As x_1 and x_2 are binary variables, then the following version of f_1 in Equation 4 is mathematically equivalent to Equation 3, and thus the QUBO formulation is correct and complete:

$$f_1 = 2x_1^2 + 5x_2^2 \quad (4)$$

Based on Equation 4, f_1 can be represented in a matrix form as follows:

$$f_1 = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (5)$$

$$f_1 = XQ_1X^t, \quad Q_1 = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix} \quad (6)$$

The matrix Q_1 is called the QUBO matrix, and it fully represents the QUBO formulation. Based on Q_1 , we can construct both the decision variables vector and the objective function. Solving this QUBO formulation gives the optimal solution for our optimization problem. If the solution includes $x_1 = 1$, the client buys one vanilla scoop; if $x_1 = 0$, he buys no vanilla scoops; similarly for x_2 . As stated before, the solution is trivial and gives $(x_1, x_2) = (0, 0)$, which simply buys nothing at the minimal cost $f_1 = 0$.

When we further complicate our problem and add one constraint that mandates buying *exactly one scoop* instead of *at most one scoop*, we have to change our QUBO formulation to encode this constraint, allowing only feasible solutions that include buying exactly one scoop, i.e., $(x_1, x_2) = (1, 0)$ or $(x_1, x_2) = (0, 1)$, and eliminating infeasible solutions that include buying zero scoops, i.e., $(x_1, x_2) = (0, 0)$, or two scoops, i.e., $(x_1, x_2) = (1, 1)$. As stated before, QUBO formulations are unconstrained by default, so constraints are enforced through a workaround by adding penalty terms in the objective function. In our example, we use the same two decision variables, x_1 and x_2 , which correspond to vanilla and chocolate scoops, and formulate the new objective function as follows:

$$f_2 = 2x_1 + 5x_2 + P(x_1 + x_2 - 1)^2, \quad P > 7 \quad (7)$$

f_2 is exactly the same as f_1 in Equation 3 with one addition penalty term $tp = P(x_1 + x_2 - 1)^2$. This penalty term penalizes only infeasible solutions, $(x_1, x_2) = (0, 0)$ or $(1, 1)$, where $tp = P$. In feasible solutions, $(x_1, x_2) = (1, 0)$ or $(0, 1)$, the penalty $tp = 0$ and does not add any cost to the objective function. Expanding Equation 7 assuming the penalty constant is arbitrarily selected as $P = 10$, f_2 can be represented in a matrix form as follows:

$$f_2 = 2x_1 + 5x_2 + 10(x_1^2 + x_2^2 + 1 + 2x_1x_2 - 2x_1 - 2x_2)$$

$$f_2 = 2x_1^2 + 5x_2^2 + 10(x_1^2 + x_2^2 + 1 + 2x_1x_2 - 2x_1^2 - 2x_2^2)$$

$$f_2 = -8x_1^2 - 5x_2^2 + 10 + 20x_1x_2$$

$$f_2 = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} -8 & 10 \\ 10 & -5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 10$$

$$f_2 = XQ_2X^t + 10, \quad Q_2 = \begin{bmatrix} -8 & 10 \\ 10 & -5 \end{bmatrix} \quad (8)$$

It is worth noting that the final form of both objective functions, f_1 in Equation 6 and f_2 in Equation 8, is exactly the same, obeying the general form $f = XQX^t + c$, with only changing the value of the QUBO matrix Q and the constant c . The constant c is negligible as it adds the same cost for all possible solutions and therefore does not impact the optimization process, so the primary outcome of any QUBO modeling is the QUBO matrix Q . The dimensionality of Q determines the number of decision variables, and its element values determine the coefficients of the objective function f . Given Q , one can construct the entire QUBO formulation exactly.

Fred Glover's tutorial [8] provides several examples and technicalities for composing QUBO formulations, such as modeling different constraints, selecting penalty values, etc. Our tutorial in [35] provides additional real-world examples with detailed explanations, and hands-on coding examples are available in [2, 3].

4.3 Hybrid Solvers

As discussed above, even though any NP-complete problem can be mapped into an instance of the Ising model (Equation 2) and its corresponding QUBO formulation, the limitations of physical quantum annealing devices restrict the types and sizes of problems that can run on them. The sparse connectivity of superconducting quantum processors requires using multiple qubits to represent a single problem variable, reducing the problem instance size that can be solved. This becomes even more problematic for constrained problems. Since the native problem solved by quantum annealers is a QUBO problem, solving constrained problems first requires transforming the constraints into additional penalty terms that need to be added to the objective function. However, this brings new issues at the hardware level: (i) enforcing equality constraints as penalty terms results in an increasing connectivity requirement (usually full connectivity); (ii) enforcing inequality constraints requires using slack variables, which need to be represented by physical qubits, further reducing the effective size of the instances that can be solved; (iii) guaranteeing the faithful mapping of optimal solutions can require fine-tuning of the penalty terms coefficients; and (iv) the required strength of the penalty terms may be beyond the dynamic range of the programmable parameters of the quantum annealer, resulting in some of the smaller coefficients in the objective function being effectively neglected. Most of these issues, which can have a negative effect on the performance of the device, also apply to other platforms like a digital annealer.

One way to get around these obstacles is to abandon the goal of a faithful mapping of optimal solutions and exploit the potential computational power of the quantum annealer in a hybrid framework. The main idea of a hybrid solver is to pair the quantum processor

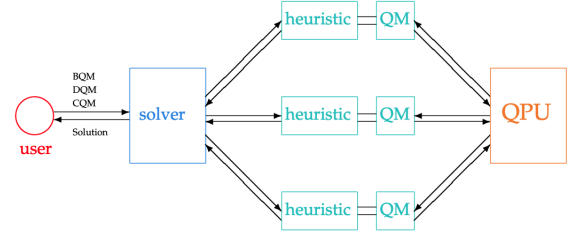


Figure 5: D-Wave hybrid solver framework [11]

with a classical heuristic in an iterative fashion, where this heuristic explores the search space and the output of quantum queries sent to the quantum module (QM) is used to suggest more promising areas of the solution space that the classical heuristic can further explore. A front-end takes the problem description as input and runs multiple threads of classical heuristics; the QM generates native QUBO problems that can be solved by the Quantum Processor Unit (QPU) using quantum annealing; the results are passed back to the front-end that outputs the best solution after a user-provided time limit.

Figure 5 shows a schematic of the D-Wave Quantum Inc. hybrid solver framework, a Python framework to build classical-quantum hybrid solvers. They also provide four user-ready pre-built hybrid solvers: the Binary Quadratic Model (BQM) [16], the Discrete Quadratic Model (DQM) [12], the Constrained Quadratic Model (CQM) [17], and the nonlinear-program (NL) solver [18]. The BQM has the same QUBO features of using binary variables and unconstrained quadratic objective functions, yet it incorporates classical computing power in its computations as in Figure 5. The DQM adds a new feature that enables integer variables, rather than being limited to binary variables. This enables modeling of more complex problems with fewer variables, thereby overcoming some of the limitations discussed earlier. DQM has limited support for constraints, supporting only one-hot constraints natively on its discrete-valued variables. The CQM further enhances DQM's capabilities by supporting binary, integer, and real-valued variables and natively handling one-hot, equality, and inequality constraints, with up to 100K constraints on 500K variables. This is the most advanced quadratic model among all D-Wave hybrid solvers and can encode a variety of real-world problems [11]. The nonlinear-program (NL) hybrid solver is the first to support variables of composite types, specifically, sets, lists, and disjoint lists, and still supports linear, quadratic, inequality, and equality constraints natively. This allows NL solvers to encode linear, quadratic, and convex optimization problems of higher orders. Our tutorial [35] give further explanation and details on existing hybrid solvers and building new customized hybrid solvers.

5 Summary and Resources

Quantum computing provides a principled way to approach the combinatorial and constraint-rich optimization problems that arise throughout spatial computing, spanning both the quantum gate model and quantum annealing paradigms. This tutorial provides an in-depth introduction for both quantum gate model and quantum

annealing. The tutorial material is available through the Q-Spatial Workshop online outlets [34, 48, 49]. The tutorial has three parts: quantum computing fundamentals [46], the quantum gate model and representative circuits [47], and quantum annealing with QUBO modeling and hybrid classical-quantum solvers [35]. We additionally provide a dedicated hands-on session on programming the D-Wave quantum annealer [2, 3]. Appendices A and B provide further pointers to resources and tools that facilitate easier quantum modeling and further exploration for use cases that span the intersection of quantum computing and spatial computing.

References

- [1] D. Aharonov, W. van Dam, J. Kempe, Z. Landau, S. Lloyd, and O. Regev. Adiabatic quantum computation is equivalent to standard quantum computation. In *45th Annual IEEE Symposium on Foundations of Computer Science*, pages 42–51, 2004.
- [2] Yunhan Chang and Amr Magdy. Quantum Lab: Hands-on Demonstrations. <https://qspatial.github.io/#quantumlab>, 2025.
- [3] Yunhan Chang and Amr Magdy. Quantum Lab: Hands-on Session on D-Wave Quantum Annealer. <https://www.youtube.com/watch?v=avMzfNf0zMU>, 2025.
- [4] Yunhan Chang, Amr Magdy, Federico Spedalieri, and Ibrahim Sabek. Spatial regionalization: A hybrid quantum computing approach. In *Proceedings of the 33rd ACM International Conference on Advances in Geographic Information Systems*, pages 1130–1133, 2025.
- [5] Yunhan Chang, Amr Magdy, and Federico M. Spedalieri. Quantum modeling of spatial contiguity constraints. In *ACM SIGMOD Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data)*, 2025.
- [6] Arnab Das and Bikas K. Chakrabarti. Colloquium: Quantum annealing and analog quantum computation. *Rev. Mod. Phys.*, 80:1061–1081, Sep 2008.
- [7] A. B. Finnila, M. A. Gómez, C. Sebenik, C. Stenson, and J. D. Doll. Quantum annealing: a new method for minimizing multidimensional functions. *Chemical Physics Letters*, 219(5-6):343–348, 1994.
- [8] Fred Glover, Gary Kochenberger, Rick Hennig, and Yu Du. Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Annals of Operations Research*, 314(1):141–183, 2022.
- [9] Fiona Hanington. What is the ocean sdk? <https://support.dwavesys.com/hc/en-us/articles/360003680694-What-Is-the-Ocean-SDK>.
- [10] Fiona Hanington. What is the zephyr topology? <https://support.dwavesys.com/hc/en-us/articles/6820989477911-What-Is-the-Zephyr-Topology>.
- [11] Hybrid Solver for Constrained Quadratic Models [WhitePaper]. https://www.dwavesys.com/media/rldh2ghw/14-1055a-a_hybrid_solver_for_constrained_quadratic_models.pdf, 2021.
- [12] D-Wave Quantum Inc. Discrete Quadratic Models (DQM) Documentation. https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_dqm_properties.html.
- [13] D-Wave Quantum Inc. Ocean SDK, Python Documentation. <https://docs.dwavequantum.com/en/latest/ocean/>.
- [14] D-Wave Quantum Inc. QUBOs and Ising Models. https://docs.dwavequantum.com/en/latest/quantum_research/qubo_ising.html.
- [15] D-Wave Quantum Inc. Topologies. https://docs.dwavequantum.com/en/latest/quantum_research/topologies.html.
- [16] DWave Quantum Inc. Binary Quadratic Models (BQM). https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_bqm_properties.html.
- [17] DWave Quantum Inc. Constrained Quadratic Models (CQM). https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_cqm_properties.html.
- [18] DWave Quantum Inc. DWave Nonlinear Solver. https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_nl_properties.html.
- [19] Nouhaila Innan, Mohamed Bennai, Muhammad Shafique, and Moustafa Youssef. Quantum Computing Research in the Arab World. *Communications of the ACM*, 68(10):62–65, 2025.
- [20] Quantum Insider. 8 Companies That Work with Quantum Annealing Technology: A Powerful Approach to Optimization. <https://thequantuminsider.com/2024/06/10/8-companies-that-work-with-quantum-annealing-technology-a-powerful-approach-to-optimization/>.
- [21] Marin Ivezić. Quantum Computing Modalities: Quantum Annealing (QA) & Adiabatic QC (AQC). <https://postquantum.com/quantum-modalities/annealing-adiabatic/>.
- [22] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose. Quantum annealing with manufactured spins. *Nature*, 473(7346):194–198, 2011.
- [23] Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse Ising model. *Phys. Rev. E*, 58:5355–5363, Nov 1998.
- [24] Andrew D King, Juan Carrasquilla, Jack Raymond, Isil Ozfidan, Evgeny Andriyash, Andrew Berkley, Mauricio Reis, Trevor Lanting, Richard Harris, Fabio Altomare, et al. Observation of topological phenomena in a programmable lattice of 1,800 qubits. *Nature*, 560(7719):456–460, 2018.
- [25] Andrew D. King, Jack Raymond, Trevor Lanting, Rami Harris, Alessandro Zucca, Francesco Altomare, et al. Quantum critical dynamics in a 5,000-qubit programmable spin glass. *Nature*, 617(7959):61–66, 2023.
- [26] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [27] Yaroslav Koshka and M. A. Novotny. Comparison of D-Wave Quantum Annealing and Classical Simulated Annealing for Local Minima Determination. In *Proceedings of the 2020 IEEE International Conference on Rebooting Computing (ICRC)*, pages 1–8, 2020.
- [28] Hanwen Liu, Abhishek Kumar, Federico Spedalieri, and Ibrahim Sabek. Hybrid quantum-classical optimization for bushy join trees. In *Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications*, pages 20–24, 2025.
- [29] Hanwen Liu, Federico Spedalieri, and Ibrahim Sabek. A Demonstration of Q2O: Quantum-Augmented Query Optimizer. *Proceedings of the VLDB Endowment*, 18(12):5439–5443, 2025.
- [30] Amr Magdy. Quantum optimization realized: Quantum annealing for combinatorial spatial problems in practice. In *ACM SIGSPATIAL Workshop on Quantum Computing and Quantum-Inspired Technologies for Spatial Data Systems and Applications (Q-Spatial)*, 2025.
- [31] Ari Mizel, D. A. Lidar, and M. W. Mitchell. Erratum: Simple proof of equivalence between adiabatic quantum computation and the circuit model [phys. rev. lett. 99, 070502 (2007)]. *Phys. Rev. Lett.*, 127:139901, Sep 2021.
- [32] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [33] Catherine Potts. Quantum Optimization Realized: D-Wave Quantum Computing. <https://youtu.be/MmdSZngmpAU>, 2025.
- [34] Q-Spatial. Quantum Spatial Channel. <https://www.youtube.com/@QuantumSpatial>, 2025.
- [35] Ibrahim Sabek and Amr Magdy. Introduction to Quantum Annealing. <https://www.youtube.com/watch?v=tnf0eOgyWk>, 2025.
- [36] Pranshi Saxena, Ibrahim Sabek, and Federico Spedalieri. Constrained quadratic model for optimizing join orders. In *ACM SIGMOD Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data)*, pages 38–44, 2024.
- [37] Ahmed Shokry and Moustafa Youssef. Quantum computing for location determination. *arXiv preprint arXiv:2106.11751*, 2021.
- [38] Ahmed Shokry and Moustafa Youssef. Towards quantum computing for location tracking and spatial systems. In *Proceedings of the 29th international conference on advances in geographic information systems*, pages 278–281, 2021.
- [39] Ahmed Shokry and Moustafa Youssef. Device-independent quantum fingerprinting for large scale localization. In *2022 20th Mediterranean Communication and Computer Networking Conference (MedComNet)*, pages 208–215. IEEE, 2022.
- [40] Ahmed Shokry and Moustafa Youssef. Qloc: A realistic quantum fingerprint-based algorithm for large scale localization. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 238–246. IEEE, 2022.
- [41] Ahmed Shokry and Moustafa Youssef. A quantum algorithm for rf-based fingerprinting localization systems. In *2022 IEEE 47th Conference on Local Computer Networks (LCN)*, pages 18–25. IEEE, 2022.
- [42] Ahmed Shokry and Moustafa Youssef. Quantum fingerprinting for heterogeneous devices localization. *Computer Communications*, 204:43–51, 2023.
- [43] Ahmed Shokry and Moustafa Youssef. An efficient quantum euclidean similarity algorithm for worldwide localization. *arXiv preprint arXiv:2407.14680*, 2024.
- [44] Ahmed Shokry and Moustafa Youssef. A quantum access points selection algorithm for large-scale localization. In *2024 IEEE 49th Conference on Local Computer Networks (LCN)*, pages 1–7. IEEE, 2024.
- [45] The D-Wave Advantage System [WhitePaper]. https://www.dwavesys.com/media/3xvdiqcn/14-1058a-a_advantage_processor_overview.pdf, 2022.
- [46] Moustafa Youssef. Fundamentals of Quantum Computing. <https://www.youtube.com/watch?v=cuOtdSVjg90>, 2025.
- [47] Moustafa Youssef. Introduction to Quantum Circuits (Quantum Gate Model). <https://www.youtube.com/watch?v=jyR-fK1vWc>, 2025.
- [48] Moustafa Youssef, Ibrahim Sabek, and Amr Magdy. Q-Spatial Workshop 2025. <https://qspatial.github.io/#recordings>, 2025.
- [49] Moustafa Youssef, Ibrahim Sabek, and Amr Magdy. Q-Spatial Workshop 2025. <https://www.youtube.com/playlist?list=PLd1EcSmWzPFd5jDy8ul2nmq9CEPM9FuMQ>, 2025.
- [50] Yousef Zook, Ahmed Shokry, and Moustafa Youssef. A quantum fingerprinting algorithm for next generation cellular positioning. *IEEE Journal on Selected Areas in Communications*, 42(1):93–102, 2023.
- [51] Yousef Zook, Ahmed Shokry, and Moustafa Youssef. An efficient quantum binary-neuron algorithm for accurate multi-story floor localization. In *2024 14th International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, pages 1–6. IEEE, 2024.

Appendix

A Quantum Spatial Resources

This appendix gives pointers to entities, resources, projects, and use cases that relate quantum technologies to spatial computing. This list is not extensive, and it is under continuous development. One can use these pointers as starting points for further investigations into topics and use cases of interest.

A.1 Entities

- **NASA Quantum Artificial Intelligence Laboratory (QuAIL).** <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/nasa-quail/>
- **IEEE GRSS's Quantum Earth Science and Technology Technical Committee (QUEST-TC).** <https://www.grss-ieee.org/technical-committees/quantum-earth-science-and-technology-quest/>

A.2 Resources

- **Quantum computing for space applications: a selective review and perspectives.**
Authors: Torta, P., Casati, R., Bruni, S., Mandarino, A. and Prati, E., 2025.
Journal: EPJ Quantum Technology, 12(1), p.66.
<https://epjquantumtechnology.springeropen.com/articles/10.1140/epjqt/s40507-025-00369-8>
- **Tutorial on “Quantum Computing for Earth Observation” – IEEE’s-GRSS.**
<https://www.grss-ieee.org/events/tutorial-on-quantum-computing-for-earth-observation/>
- **Quantum Computing for Earth Observation Use Cases Definition and Design Report.**
<https://eo4society.esa.int/wp-content/uploads/2024/01/QC4EO-WP1-D1-Use-case-definition-and-design-report.pdf>
- **Quantum Computing for Earth Observation (QC4EO).**
https://www.grss-ieee.org/wp-content/uploads/2024/08/Bertrand_Le_Saux_HDCRS_2024.pdf
- **Physics Guided and Quantum Artificial Intelligence for Earth Observation.**
<https://www.2024.ieeeigarss.org/tutorials.php#tut13>
- **GIS and Quantum, by Jessi McPhaul.**
https://rstudio-pubs-static.s3.amazonaws.com/1286147_01beae6410cd48df99d7d68460424d15.html
- **Opportunities and challenges of quantum computing for climate modeling.**
<https://doi.org/10.1017/eds.2025.10010>

A.3 Projects and Use Cases

- **European Space Agency (ESA) “Quantum Computing for Earth Observation” (QC4EO) Initiative.**
<https://eo4society.esa.int/projects/qc4eo-study/>
- **ESA Project: Towards Operational Quantum Computing for Earth Observation.**
<https://esastar-publication-ext.sso.esa.int/ESATenderActions/details/92141>

- **Supply chain logistics with quantum and classical annealing algorithms.**
<https://www.nature.com/articles/s41598-023-31765-8>
- **Quantum computing for traffic flow optimization.**
<https://www.volkswagen-group.com/en/press-releases/research-project-successful-volkswagen-it-experts-use-quantum-computing-for-traffic-flow-optimization-16498>
- <https://www.openqase.com/case-study/d-wave-toyota-quantum-computing-traffic-flow-optimization>
- <https://www.openqase.com/case-study/d-wave-volkswagen-traffic-optimization-and-paint-shop-scheduling>
- **Quantum Computing and GEOINT.**
<https://usgif.org/is-quantum-computing-poised-to-transform-geoint/>
- **New Quantum Capabilities for Naval Warfare Centers.**
<https://www.nrl.navy.mil/Media/News/Article/3263682/new-quantum-capabilities-for-naval-warfare-centers/>
- <https://www.nrl.navy.mil/Our-Work/Areas-of-Research/Quantum-Research/>
- <https://defensescoop.com/2023/12/05/navy-eyes-quantum-tech-to-simulate-warfighting-probabilities/>
- **ESA and European Commission to build quantum-secure space communications network.**
https://www.esa.int/Applications/Connectivity_and_Secure_Communications/ESA_and_European_Commission_to_build_quantum-secure_space_communications_network
- **Study Explores Hybrid Quantum Algorithms for Improved Weather Prediction, Climate Modeling.**
<https://thequantuminsider.com/2024/10/21/study-explores-hybrid-quantum-algorithms-for-improved-weather-prediction-climate-modeling/>
- **Quantum Computing and Climate Modeling.**
<https://quantumzeitgeist.com/quantum-computing-for-climate-change-how-can-it-help/>
- <https://thequantuminsider.com/2024/10/21/study-explores-hybrid-quantum-algorithms-for-improved-weather-prediction-climate-modeling/>
- <https://doi.org/10.1017/eds.2025.10010>
- **Quantum Navigation and GPS-free Positioning Systems** <https://www.weforum.org/stories/2024/07/what-is-quantum-navigation-earth-observation/>

B Auto QUBO Software Tools

This appendix gives pointers to software tools that enable users to encode optimization problems into Quadratic Unconstrained Binary Optimization (QUBO) formulations. This list is not extensive, and it is under continuous development.

- **Automated QUBO Generator - Fujitsu Research**
<https://github.com/FujitsuResearch/autoqubo>
- **AutoQubo: data-driven automatic QUBO generation**
<https://dl.acm.org/doi/10.1145/3520304.3533965>
- **AutoQUBO v2: Towards Efficient and Effective QUBO Formulations for Ising Machines**
<https://dl.acm.org/doi/10.1145/3583133.3590662>
- **LLM-QUBO: An End-to-End Framework for Automated QUBO Transformation from Natural Language Problem Descriptions**

<https://ojs.aaai.org/index.php/AAAI-SS/article/view/36913/39051>

- **Manual vs. Automated QUBO Formulations for Flow Shop Scheduling: A Comparative Study on D-Wave and InfinityQ**

<https://dl.acm.org/doi/10.1145/3712255.3734364>

- **Julia ToQUBO Package**

<https://github.com/JuliaQUBO/ToQUBO.jl>

- **MQT Quantum Auto Optimizer**

<https://mqt.readthedocs.io/projects/qao/en/latest/>